

Zinger Engineering Handbook

Technical specification for the public Core repository

Zinger Project — v1.1.0

August 2026

Contents

1	Introduction	3
1.0.1	Repository scope	3
1.0.2	Runtime requirements	3
2	Architecture	4
2.1	Process topology	4
2.2	Scan pipeline (four phases)	4
2.3	Persistence	4
3	Three strategy regimes	5
3.0.1	Governor forbidden keys	5
3.0.2	Dual-layer regime detection	5
4	Arbitrage engine	6
4.1	Gap definition	6
4.2	Two-stage gate (<code>arbEngine.ts</code>)	6
5	Directional trading and sizing	7
5.1	Kelly criterion	7
5.2	Exit matrix	7
6	ML regime layer	8
6.1	Statistical jump model	8
6.2	Emission pipeline	8
6.3	Alpha fusion	8
7	Risk governance	9
7.1	Authority hierarchy	9
7.2	Drawdown circuit breaker	9
7.3	Live edge gate	9
8	Public HTTP API	10
9	Configuration	11
9.1	Environment variables	11
9.2	Mode profiles	11
10	Operations	12
10.1	Quick start	12
10.2	Checks	12
10.3	Regime refresh (production)	12
11	Testing	13

12 Related repositories	14
A License and security	15
B Building this handbook	16

Chapter 1

Introduction

Zinger is a Polymarket trading system for BTC and ETH up/down prediction markets. It runs in **paper** or **live** mode, exposes an operator dashboard at `/poly`, optional Telegram control, a public signal API, and Python ML helpers for regime detection.

1.0.1 Repository scope

This public tree ships **Core only**:

- Express API and Polymarket bot (`src/`)
- Operator dashboard (`frontend/`)
- ML training and regime scripts (`ml/`)
- Public signal playground package (`public-api/`)
- Unit and performance tests (`tests/`)

The **Pilot consumer app** (wallet product) is maintained in the private `zinger-pilot` repository and is intentionally excluded from this tree.

1.0.2 Runtime requirements

- Node.js ≥ 20 (22 recommended)
- TypeScript executed via `tsx`
- Polygon PoS for live CTF settlement
- Optional Python 3.12 + `ml/.venv` for regime emission

Chapter 2

Architecture

2.1 Process topology

```
index.ts --> src/server.ts
             |-- src/polymarket/*    (bot, scan, engines, store)
             |-- src/ai/*            (governor, optimizer)
             |-- src/telegram/*
             |-- frontend/dist       (operator UI at /poly)
             |-- public-api          (optional playground)
```

2.2 Scan pipeline (four phases)

The bot no longer runs a monolithic loop. Each scan pass is:

1. **Cycle** — window timing, market discovery, eligibility
2. **Inputs** — signals, fusion context, book depth
3. **Exits** — TP/SL, trailing, partials, settlement
4. **Orchestrator** — directional entries, arb packages, telemetry

Modules live under `src/polymarket/scan/`.

2.3 Persistence

State is stored in SQLite WAL mode (`data/zinger.db`, table `docs`): configuration, trades, governor state, ML signals, and equity curves. Runtime `data/` is gitignored; use `ZINGER_DATA_DIR` in tests.

Chapter 3

Three strategy regimes

The AI governor (`src/ai/governor.ts`) selects one of three profiles:

Profile	When	Behaviour
<code>trend-ride</code>	ADX trending, ATR moderate	Wider TP, ride winners, mid-band entries
<code>scalp</code>	ADX chop / range	Tighter TP/SL, faster exits
<code>arb-only</code>	High ATR or ML high-vol	Directional muted; CLOB arb only

3.0.1 Governor forbidden keys

The governor may turn arb *on or off* but must never write arb *thresholds*: `minArbGap`, `arbMinMarginPct`, `maxArbPackages`, `arbBankrollFrac`, `arbMaxUsd`. Those are stripped inside `applyProfile`. The fee-aware gate in `arbEngine.ts` owns economics.

3.0.2 Dual-layer regime detection

1. **Heuristic** — ADX + ATR on BTC/ETH spot → trend-ride / scalp / arb-only
2. **ML overlay** — statistical jump model (`ml/regime_emit.py`) → high-vol forces arb-only; calm defers to ADX (keeps `scalp` reachable)

Shared reader: `src/polymarket/regimeSignal.ts` (6h freshness gate).

Chapter 4

Arbitrage engine

4.1 Gap definition

For complementary UP/DOWN asks:

$$\text{Combined} = \text{Ask}_{\text{UP}} + \text{Ask}_{\text{DOWN}}, \quad \text{Gap} = 1 - \text{Combined}$$

4.2 Two-stage gate (`arbEngine.ts`)

1. **Fee break-even (mandatory):** $\text{Gap} > \text{arbBreakEvenGap}(\text{up}, \text{down}) + \text{arbMinMarginPct}$
2. **Operator floor:** $\text{Gap} \geq \text{minArbGap}$ (runs after; cannot admit unprofitable trades)

Packages are atomic: both legs fill or the engine unwinds. Orphan legs are valued at market mid, not a fixed \$0.50 placeholder.

Chapter 5

Directional trading and sizing

5.1 Kelly criterion

Fractional Kelly in `kelly.ts`:

$$f^* = \frac{bp - q}{b}, \quad \text{size} = \text{bankroll} \cdot \min(\text{maxPositionPct}, f^* \cdot \text{kellyFraction})$$

Optional vol tilt: `realizedVol / calmBaseline` from the jump model shrinks size in high-vol states.

5.2 Exit matrix

Exit	Trigger	Role
FAST SL	$\text{price} \leq \text{entry} \cdot (1 - \text{slPct})$	Hard stop
ADAPTIVE SL	underwater + confidence fade	Tighten stop
PARTIAL TP	fraction of TP target hit	De-risk into momentum
TRAILING STOP	retrace from high water mark	Lock trend gains
DISASTER SL	underdog deep drawdown	Hold-to-settle breathing room
SETTLE	window end	Binary resolution

Chapter 6

ML regime layer

6.1 Statistical jump model

`ml/regime_jump.py` fits a two-state penalized clustering model on downside deviation and Sortino features (z-scored). Penalty default 1.0 (override: `ZINGER_REGIME_PENALTY`).

6.2 Emission pipeline

```
ml/regime_refresh.py    # cron: refresh OHLCV + emit
ml/regime_emit.py       # writes regime_signal.json to SQLite store
regimeSignal.ts         # single consumer for governor + alpha fusion
```

6.3 Alpha fusion

`src/polymarket/alphaFusion.ts` blends signal, book imbalance, and regime context before directional scoring. Book depth is captured in `bot.ts` into `booksForFusion`.

Chapter 7

Risk governance

7.1 Authority hierarchy

1. **Operator** — `forceArbOnly`, live edge gate overrides
2. **Guardrails** — drawdown breaker, `arbOnlyUntilEdge`
3. **Automation** — governor profiles, LLM optimizer (bounded)

7.2 Drawdown circuit breaker

When equity falls $\geq$ `governorDrawdownPct` off peak, directional trading locks and the engine favours arb-only until recovery or operator reset.

7.3 Live edge gate

`evaluateEdgeGate` requires sufficient directional paper closes with positive expectancy and Kelly before live directional is allowed. Arb can still be enabled separately.

Chapter 8

Public HTTP API

Core exposes (via `publicPredictions.ts`):

Endpoint	Purpose
GET <code>/api/v1/predictions</code>	Full signal + market snapshot
GET <code>/api/v1/predictions/stream</code>	SSE prediction stream
GET <code>/api/v1/paper</code>	Published bot paper book
GET <code>/api/v1/bot-paper</code>	Same book (playground default)
GET <code>/api/v1/markets</code>	Active Polymarket windows
GET <code>/api/v1/market/:slug</code>	Single market + book
GET <code>/api/v1/charts/spot</code>	BTC/ETH spot history
GET <code>/api/v1/health</code>	API + bot health

Pilot wallet routes (`/api/v1/pilot/*`) are **not** part of public Core.

The standalone `public-api/` package mirrors a trimmed signal surface for embedding and documentation.

Chapter 9

Configuration

9.1 Environment variables

Variable	Purpose
AUTH_PASSWORD	Dashboard login
OPENROUTER_API_KEY	Optional LLM governor
TELEGRAM_BOT_TOKEN	Optional Telegram bot
CLOB_PROXY_URL	Egress for live order writes
ZINGER_DATA_DIR	Runtime data directory
ZINGER_REGIME_PENALTY	Jump-model penalty (default 1.0)

9.2 Mode profiles

`poly_config.json` stores per-mode (**paper**, **live**) profiles: bankroll, confidence bands, arb flags, governor enablement, Kelly fraction, etc.

Chapter 10

Operations

10.1 Quick start

```
git clone https://github.com/David-glitch/Zinger.git
cd Zinger && npm install
cd frontend && npm install && npm run build && cd ..
cp .env.example .env
npm start
# http://localhost:3000/poly
```

10.2 Checks

```
npm run typecheck
npm test
npm run test:perf
npm run ci
```

10.3 Regime refresh (production)

Schedule `ml/regime_refresh.py` every 2h so the ML overlay stays inside the 6h freshness gate. Governor falls back to ADX/ATR if stale.

Chapter 11

Testing

- **252 unit tests** across 23 files (invariants, arb, governor, regime reachability)
- **4 perf tests** — hot-path budgets for fees, parsers, Kelly, auth
- Tests isolate data via `ZINGER_DATA_DIR` per worker

Key suite: `tests/unit/regimeReachability.test.ts` pins all three governor profiles reachable.

Chapter 12

Related repositories

Repository	Role
David-glitch/Zinger	This public Core monorepo
David-glitch/zinger-core	Apache-2.0 extraction (sync target)
David-glitch/zinger-pilot	Private consumer wallet app

Appendix A

License and security

Public Core is MIT-licensed. Report vulnerabilities per `SECURITY.md`. Never commit `.env`, wallets, or `data/` contents.

Appendix B

Building this handbook

```
cd docs/handbook  
tectonic zinger-handbook.tex  
# Output: zinger-handbook.pdf
```